

RAAS PAL

Robot Recommendation System

Architecture & Design Specification

Version 1.0 | May 2026
Stack: Java / Spring Boot • Next.js • PostgreSQL • Claude AI

This document covers system architecture, role design, entity design,
API endpoints, security model, and package structure.

1. System Overview

Raas Pal's Robot Recommendation System is a web-based platform that enables the Raas Pal team to analyze customer requirements (industry, site conditions, floor types, cleaning needs, coverage area, and budget) against a structured robot catalog, then generate recommendations with supporting reports. The system launches with cleaning robots and is designed to extend to other robot types (delivery, security, and more), each with its own specifications but sharing the same recommendation engine and scoring approach.

1.1 Technology Stack

Layer	Technology	Purpose
Backend	Java 21 + Spring Boot 3.x	REST API, business logic, security
Frontend	Next.js 14 (React)	Requirement wizard, dashboards, reports
Database	PostgreSQL 16	Primary data store
Cache	Redis	Sessions, AI response caching
AI	Claude API via Spring AI	Recommendation reasoning, NL explanations
Embeddings	OpenAI / pgvector	Semantic similarity search
Auth	Spring Security + JWT	Role-based access control
Export	JasperReports / iText	PDF recommendation reports

1.2 Development Milestones

Recommended solo-developer build order:

Phase	Scope	Outcome
1	Database schema	PostgreSQL tables for robots, specs, requirements
2	Data cleaning & import	Messy datasheet CSV parsed into clean, typed rows
3	Spring Boot skeleton	Entities, CRUD, OpenAPI docs
4	Recommendation engine	Hard filter, weighted scoring over 6 dimensions
5	AI integration	Spring AI + Claude explanations + pgvector nudge
6	Next.js frontend	Wizard, results, comparison view
7	Admin panel	Robot catalog CRUD, scoring weight config
8	Reports + export	PDF generation
9	Deployment	Docker Compose → AWS (ECS, RDS, S3)

Note: Phase 2 (data cleaning) is new and essential. The robot datasheets arrive as messy, sparse spreadsheets — values like “170 (machine weight) / 215 (total weight)” and many blank cells. This raw data cannot be scored directly; it must first be parsed into clean, typed, nullable fields. Nothing downstream works until this is done.

2. Role Design

2.1 Role Enum

Three roles are defined system-wide:

Role	Description	Dashboard Route
ADMIN	Full system access — users, robots, all customers, audit logs, scoring config	/dashboard/admin
RAASPAL_TEAM	Customer management, requirement intake, generate recommendations and reports — no user mgmt or audit log	/dashboard/team
CUSTOMER	Own requirements, recommendations, reports, file uploads only — data isolation enforced server-side	/dashboard/customer

2.2 Dashboard Access Logic

After a successful login the backend issues a JWT. The frontend reads the role claim and redirects automatically:

JWT Redirect Flow		
ADMIN	→ /dashboard/admin	(full system)
RAASPAL_TEAM	→ /dashboard/team	(operations)
CUSTOMER	→ /dashboard/customer	(own data only)
Backend enforces ownership — role alone is not enough for CUSTOMER. Every CUSTOMER request is verified: requirement.customer_profile_id == currentUser.customerProfileId		

2.3 JWT Claims Structure

JWT Payload	
{	
"sub":	"user@email.com",
"userId":	"uuid",
"role":	"CUSTOMER",
"name":	"Jane Doe",
"customerProfileId":	"uuid", // only present for CUSTOMER role
"iat":	1234567890,

```
"exp": 1234567890
}
```

The `customerProfileId` claim embedded in the token lets the backend enforce data isolation for customers without an extra database lookup on every request.

3. Entity Design

3.1 User

All three roles share a single User table and the same `/api/v1/auth/login` endpoint. ADMIN and RAASPAL_TEAM are internal users with no profile extension. CUSTOMER users must have a corresponding CustomerProfile row.

Column	Type	Constraints
id	UUID	PK, auto-generated
email	VARCHAR	Unique, not null
password	VARCHAR	bcrypt hashed, not null
full_name	VARCHAR	Not null
role	ENUM(ADMIN, RAASPAL_TEAM, CUSTOMER)	Not null
is_active	BOOLEAN	Default true
created_at	TIMESTAMP	Auto
updated_at	TIMESTAMP	Auto

3.2 CustomerProfile

Created atomically with the User when RAASPAL_TEAM onboards a customer. One profile per customer user (1:1 with User).

Column	Type	Constraints
id	UUID	PK
user_id	UUID	FK → User (unique)
company_name	VARCHAR	Not null
industry	VARCHAR	
contact_phone	VARCHAR	
address	TEXT	
notes	TEXT	Internal notes by RAASPAL_TEAM
created_by	UUID	FK → User (the team member who onboarded)
created_at	TIMESTAMP	Auto
updated_at	TIMESTAMP	Auto

3.3 Requirement

The central intake entity, capturing what the customer needs. Ownership is anchored to CustomerProfile. All role-based ownership checks use: requirement.customer_profile_id == currentUser.customerProfileId.

The fields below reflect the cleaning-robot domain. The structured fields drive the hard filter and scoring; the free-text description feeds the AI semantic matching. As Raas Pal adds other robot types (delivery, security), each type will have its own requirement profile, but the ownership and status structure stays identical.

Column	Type	Notes
id	UUID	PK
customer_profile_id	UUID	FK → CustomerProfile (ownership anchor)
robot_type	ENUM(CLEANING, DELIVERY, SECURITY, ...)	Which catalog to match against
title	VARCHAR	Short name for the request
description	TEXT	Free-text needs (feeds AI matching)
environment	ENUM(INDOOR, OUTDOOR, CLEANROOM, HAZARDOUS)	Hard constraint
cleaning_functions	TEXT[] / join table	Needed: sweep, scrub, mop, vacuum
floor_types	TEXT[] / join table	Customer's floor surfaces
min_passable_width_mm	INT	Narrowest doorway / corridor
coverage_area_sqm	INT	Area to clean per shift
budget_band	ENUM(LOW, MODERATE, HIGH)	Soft constraint for scoring
priority_notes	TEXT	Additional context
status	ENUM(DRAFT, SUBMITTED, IN_PROGRESS, COMPLETED)	Workflow state
created_by	UUID	FK → User
created_at	TIMESTAMP	Auto
updated_at	TIMESTAMP	Auto

3.4 Robot and RobotSpec

The robot catalog is split into two tables. Robot holds identity and the few universal fields; RobotSpec holds the large set of type-specific technical specs. Every spec column is nullable — a blank means “unknown” (e.g. a spec not yet measured on an untested robot), never “zero” or “bad.”

Robot (identity + universal fields):

Column	Type	Notes
id	UUID	PK

Column	Type	Notes
brand	VARCHAR	e.g. Gausium, PUDU, KEENON
model	VARCHAR	e.g. CENOBOT L3
robot_type	ENUM(CLEANING, DELIVERY, SECURITY)	Which catalog it belongs to
test_status	ENUM(DRAFT, UNDER_TESTING, VERIFIED)	Distinguishes “not yet tested” from “verified”
price_band	ENUM(LOW, MODERATE, HIGH)	Coarse price for budget scoring
image_url	VARCHAR	S3 link to product image
datasheet_url	VARCHAR	S3 link to datasheet PDF
created_at	TIMESTAMP	Auto

RobotSpec (cleaning-type specs — all nullable):

Group	Example columns (parsed & typed)	Used for
Capability	cleaning_functions[], cleaning_efficiency_sqm_h	Capability match
Size & access	length_mm, width_mm, height_mm, min_passable_width_mm, slope_angle	Size & access fit
Coverage	tank_clean_l, tank_waste_l, battery_work_hr, speed_ms	Coverage capacity
Floor support	~30 boolean floor-type flags (marble, vinyl, carpet...)	Floor suitability
Environment	is_indoor, is_outdoor, ip_rating, hepa	Environment fit
Operational	noise_db, navigation_type, auto_dock, charge_time	Operational quality

These ~50 raw columns are parsed once during the data-cleaning import (Phase 2) and grouped into the six scoring dimensions described in Section 7.

3.5 Other Entities (unchanged)

The following entities require no structural change. Ownership of recommendations and reports is derived transitively through the requirement's customer_profile_id:

Entity	Purpose
Recommendation	A recommendation session generated for a requirement
RecommendationItem	Individual robot ranked within a recommendation, with total score
Score	Per-dimension score breakdown for each item (audit trail)
FileUpload	Uploaded documents linked to requirements or reports
Report	Generated PDF report record
AuditLog	Immutable system event log (ADMIN access only)

4. Backend Package Structure

Spring Boot project root: com.raaspal.robotrecommendation

Package Layout

```
com.raaspal.robotrecommendation
|
├─ auth/
|   ├── controller/    AuthController
|   ├── service/       AuthService, TokenService
|   ├── dto/           LoginRequest, LoginResponse, TokenRefreshRequest
|   └─ security/       JwtFilter, JwtUtil, SecurityConfig,
|                       UserDetailsServiceImpl, JwtClaims
|
├─ user/
|   ├── controller/    UserController
|   ├── service/       UserService
|   ├── repository/    UserRepository
|   ├── entity/        User
|   └─ dto/            UserResponse, CreateUserRequest,
|                       UpdateUserRequest, ChangePasswordRequest
|
├─ customer/
|   ├── controller/    CustomerController
|   ├── service/       CustomerService (creates User + CustomerProfile
atomically)
|   ├── repository/    CustomerProfileRepository
|   ├── entity/        CustomerProfile
|   └─ dto/            CreateCustomerRequest, CustomerProfileResponse,
|                       UpdateCustomerProfileRequest
|
├─ robot/              RobotController, RobotService, Robot, RobotSpec
├─ requirement/        RequirementController, RequirementService,
Requirement
├─ recommendation/    RecommendationController, Recommendation,
RecommendationItem
├─ scoring/            ScoringService, ScoreEngine, Score
├─ ai/                 AiRecommendationService, EmbeddingService,
PromptBuilder
```

└─ file/	FileController, FileService, FileUpload
└─ report/	ReportController, ReportService, Report
└─ audit/	AuditLogController, AuditLogService, AuditLog
└─ common/	
└─ exception/	GlobalExceptionHandler, ResourceNotFoundException
└─ response/	ApiResponse<T>, PagedResponse<T>
└─ enums/	Role, RequirementStatus, RecommendationStatus,
Environment	
└─ audit/	AuditAspect (AOP listener)
└─ security/	CurrentUser annotation, SecurityUtil
	(helper to extract userId / customerProfileId from
JWT)	

5. Security Configuration

5.1 Access Rules Summary

Tier	Endpoints	Rule
Public	POST /auth/login, POST /auth/refresh	No token required
Authenticated	POST /auth/logout, GET+POST /files/**	Any valid JWT
ADMIN only	GET/DELETE /users/**, /audit-logs/**, DELETE /robots/{id}, DELETE /recommendations/{id}	ADMIN role
ADMIN + TEAM	POST/PUT /robots/**, POST/GET /customers/**, POST /recommendations, POST /reports	ADMIN or RAASPAL_TEAM
All roles (owned)	CRUD /requirements/**, GET /recommendations/**, GET /reports/**	Role + ownership check in service layer

5.2 Customer Onboarding Flow

Customers are created by RAASPAL_TEAM — they do not self-register. The server creates both User and CustomerProfile atomically in one transaction:

Onboarding API Call — POST /api/v1/customers

Request body (sent by RAASPAL_TEAM):

```
{
  "email": "client@company.com",
  "fullName": "Jane Doe",
  "temporaryPassword": "...",
  "companyName": "Acme Corp",
  "industry": "Automotive",
  "contactPhone": "...",
  "address": "..."
}
```

Server atomically:

1. Creates User (role = CUSTOMER, is_active = true)
2. Creates CustomerProfile linked to that User
3. Returns both User and CustomerProfile in response

Customer Login — POST /api/v1/auth/login

Request: { email, password }

Response: JWT containing role=CUSTOMER, customerProfileId=<uuid>

Frontend: redirects to /dashboard/customer

6. API Endpoint Permissions

Base path: /api/v1 | Format: JSON | Auth: Bearer JWT

6.1 Auth — /api/v1/auth

Method	Path	Access
POST	/login	Public — all roles
POST	/refresh	Public
POST	/logout	Authenticated

6.2 Users — /api/v1/users

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/	Create any user	—	—
GET	/	All users	—	—
GET	/id	Any	—	Self only
PUT	/id	Any	—	Self only
PUT	/id/password	Any	Self	Self
PUT	/id/role	Yes	—	—
DELETE	/id	Yes	—	—

6.3 Customers — /api/v1/customers

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/	Create customer	Create customer	—
GET	/	All profiles	All profiles	—
GET	/id	Yes	Yes	Own profile
PUT	/id	Yes	Yes	Own profile
DELETE	/id	Yes	—	—
GET	/id/requirements	Yes	Yes	Own only

6.4 Robot Catalog — /api/v1/robots

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST/PUT	/, /id, /specs, /image, /datasheet	Yes	Yes	—

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
DELETE	/id	Yes	—	—
GET	/, /id, /search	Yes	Yes	Yes (view)

6.5 Requirements — /api/v1/requirements

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/	Yes (any customer)	Yes (any customer)	Own profile only
GET	/	All	All	Own only (auto-filtered)
GET	/id	Yes	Yes	Own only
PUT	/id	Yes	Yes	Own, DRAFT only
DELETE	/id	Yes	Yes	Own, DRAFT only
POST	/id/submit	Yes	Yes	Own only

6.6 Recommendations — /api/v1/recommendations

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/	Yes	Yes	— (view only)
GET	/	All	All	Own only (auto-filtered)
GET	/id	Yes	Yes	Own only
GET	/id/scores	Yes	Yes	Own only
DELETE	/id	Yes	—	—

6.7 Reports — /api/v1/reports

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/	Yes	Yes	—
GET	/	All	All	Own only
GET	/id/download	Yes	Yes	Own only

6.8 Files — /api/v1/files

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/upload	Yes	Yes	Yes (own entity)
GET	/id/download	Yes	Yes	Own file only

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
DELETE	/id}	Yes	Yes	Own file, if not locked

6.9 Audit Logs — /api/v1/audit-logs

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
GET	/	Yes	—	—
GET	/entityType}/{entityId}	Yes	—	—

7. The Recommendation Engine

This is the core of the system — how a set of customer requirements becomes a ranked list of robots. It runs in two deterministic phases (pure code, no AI), with the AI layer adding refinement and explanation on top.

7.1 Data cleaning comes first

Robot datasheets arrive as messy, sparse spreadsheets. Real examples include weights written as “170 (machine weight) / 215 (total weight including water)”, slope angles as “5°” or “-”, and many blank cells. Before any scoring is possible, a one-time import parses each raw value into a clean, typed field. Blanks become null (“unknown”), never zero.

7.2 The reframe: ~50 specs become 6 dimensions

A customer does not care about 50 individual numbers — they care about a handful of outcomes. The ~50 raw spec columns are therefore grouped into six scoring dimensions. The scoring engine's complexity is bounded by the number of dimensions (6), not the number of columns (50+), which is why it scales cleanly as more specs or robot types are added.

Dimension	Question it answers	Built from
Capability match	Can it do the cleaning type needed?	Cleaning functions, efficiency
Size & access fit	Will it fit and move in the space?	Dimensions, passable width, slope
Coverage capacity	Can it cover the area per shift?	Efficiency, tank, battery, speed
Floor suitability	Does it handle their floors?	~30 floor-type flags
Environment fit	Indoor/outdoor/wet-rated?	Indoor, outdoor, IP, HEPA
Operational quality	Quiet, autonomous, well-navigated?	Noise, navigation, auto-dock

7.3 Phase 1 — Hard filter (pass / fail)

Each robot faces boolean checks against the hard constraints: required cleaning function, fits through the customer's narrowest doorway, correct environment, and IP rating where wet/outdoor. A robot failing any check is eliminated before scoring. This guarantees no robot that physically cannot do the job can ever be recommended, and keeps AI costs low by only processing qualified candidates. Each check is cheap and independent, so adding constraints later is trivial.

7.4 Phase 2 — Weighted scoring (rank 0–100)

Surviving robots are scored on each of the six dimensions, normalised to 0–1, then combined using weights that sum to 1.0. The weights are configurable and can differ by industry — a hospital weights Operational Quality (noise) heavily, while a warehouse weights Coverage Capacity. The result is a transparent 0–100 score with a full per-dimension breakdown stored for audit.

7.5 Handling missing data fairly

Because robots may be only partially tested, missing specs must never act like bad scores. When a spec is null, that dimension is dropped from the robot's calculation and the remaining weights are renormalised so the score stays fair. Each recommendation also carries a data-completeness figure (e.g. “based on 83% of specs”), and the robot's test_status distinguishes “not yet tested” from “not applicable.” Safety-critical constraints with unknown values are flagged as “needs verification” rather than silently passed.

7.6 The AI layer sits on top — it never decides the facts

Two AI capabilities enhance, but never replace, the deterministic engine. First, semantic matching: the customer's free-text description (e.g. “quiet for a hospital at night”) is embedded as a vector and compared via pgvector against robot descriptions, contributing a small nudge (roughly 10–20%) to the soft part of the score. Second, explanation: Claude narrates the already-decided ranking in plain language. The hard facts, constraints, and the bulk of the score remain pure deterministic code — so the system is precise, consistent, auditable, and still works even if the AI provider is unavailable.

8. Key Design Decisions

8.1 Two-phase recommendation engine

Hard filtering eliminates robots that cannot physically do the job (wrong cleaning type, won't fit, unsuitable floor or environment, over budget). Weighted scoring then ranks the survivors. This keeps results auditable and AI costs low — only qualified robots are ever embedded or ranked.

8.2 Rules for facts, AI for fuzzy

Numeric and constraint-based matching is handled by deterministic code, because for knowable facts code is exactly correct, consistent, and explainable. AI is reserved for the fuzzy free-text intent and for writing explanations. Letting an LLM score everything would be less accurate (LLMs slip on arithmetic and comparisons), non-deterministic, and unauditable — the wrong tool for a knowable problem.

8.3 Modular monolith (not microservices)

For a solo developer at this stage, a modular monolith with clean package boundaries is correct. Each package encodes the eventual microservice split if needed, but the overhead of service-to-service communication and distributed tracing is avoided.

8.4 pgvector instead of a dedicated vector database

pgvector runs inside PostgreSQL, keeping the stack simple. It handles the volume of robot embeddings required at any realistic Raas Pal scale. A separate Pinecone or Weaviate instance adds operational complexity with no benefit at this stage. AWS RDS PostgreSQL supports pgvector, so it works in production by simply enabling the extension.

8.5 Spring AI as the orchestrator

Spring AI provides a vendor-neutral abstraction over Claude and OpenAI. Provider switching or mixing requires no service logic changes.

8.6 Customers never self-register

RAASPAL_TEAM onboards customers. This ensures data quality, correct industry/company mapping, and prevents unauthorized access. The atomic creation of User + CustomerProfile in one transaction ensures the system is never in an inconsistent state.

8.7 customerProfileId in JWT

Embedding the customerProfileId in the JWT token eliminates one database lookup per request for ownership checks. The alternative (looking up CustomerProfile by userId on every call) adds latency and load. The tradeoff is that a role change requires token refresh, which is standard practice.

9. Recommended Next Steps

- **Step 1 — PostgreSQL schema:** Create the tables in Section 3 (User, CustomerProfile, Requirement, Robot, RobotSpec, Recommendation, etc.). Make every spec column nullable. Add indexes on foreign keys and enum columns.
- **Step 2 — Data cleaning import:** Write a one-time importer that parses the messy datasheet CSV into clean, typed, nullable rows. Treat blanks as null. This unblocks everything downstream.
- **Step 3 — Spring Boot skeleton:** Initialize the project with Spring Web, Data JPA, Security, and Spring AI. Create entities and repositories. Expose /api/v1/robots as a test endpoint.
- **Step 4 — Auth module:** Implement JWT generation, JwtFilter, SecurityConfig, and the three-role access rules from Section 5.
- **Step 5 — Customer onboarding:** Implement CustomerService with the atomic User + CustomerProfile creation transaction.
- **Step 6 — Requirement intake:** Build the Requirement CRUD with ownership enforcement and status transitions.
- **Step 7 — Recommendation engine:** Implement the hard filter and the weighted scorer over the six dimensions, including null-handling and renormalisation. Test thoroughly in Postman before adding any AI.
- **Step 8 — AI integration:** Add Spring AI + Claude for explanations, and pgvector for the semantic nudge. Cache AI responses in Redis.
- **Step 9 — Next.js frontend:** Build the requirement wizard, results page, and comparison view against the stable API contract.

— End of Document —